Parsing HTTP Outside the Lock: An `io_uring` ASGI Server with a C Parser and GIL Batching

Marimuthu Velayutham*

Abstract

***Abstract*—A Python web server usually parses HTTP while it holds the Global Interpreter Lock (GIL), and calls the application once per request. We built `uringpy-http`, an ASGI server whose `io_uring` transport and HTTP/1.1 parser run in C with the GIL released. The GIL is taken once per batch of requests, and each request starts the application as an eager `asyncio` task. Unmodified Starlette and FastAPI applications run on it. We compared it with Uvicorn and Granian on a four-core cloud server in 694 runs with no failures, against expectations recorded before the runs. With one worker, a request served by `uringpy-http` costs much less than the same request through Uvicorn with `httptools` and `uvloop`: throughput is 3.36× as high for a raw ASGI app, 2.41× for Starlette and 1.47× for FastAPI. With four worker processes the ratios are 2.76×, 2.14× and 1.44×. Granian's are 1.32×, 1.29× and 0.91×. Batching the GIL doubles what four worker threads of one interpreter deliver on the raw app (2.09×), and cuts voluntary context switches from 1.1 to 0.02 per request. Still, it reaches only 0.76 of four processes, short of the 0.90 we expected. Framework code holds the GIL for 21 µs per request (Starlette) and 106 µs (FastAPI), so four threads stay under the ceiling of one over that time; the threads then deliver 0.34 and 0.26 of processes. On the free-threaded build the same threads reach 0.97 (raw) and 0.84 (FastAPI) of processes. Code, raw data and scripts are public.**

***Index Terms*—io_uring, ASGI, HTTP parsing, Python, Global Interpreter Lock, web servers, free-threading**

I. Introduction

A request to a Python web application crosses three layers. The transport moves bytes on and off the socket. The HTTP parser turns those bytes into a method, a path and headers. The application turns the request into a response. In the usual deployment, Uvicorn [2] with `httptools` and `uvloop` [3], the parser is written in C (llhttp [4]). It still reports each part of the request through a Python callback, though, and the event loop runs Python code for every read and write. All of that holds the GIL, so the server's own work competes with the application for one lock. That is why Python servers scale by running one process per core.

Our earlier study [1] measured what this costs for worker threads that share one interpreter. It found that the number of times the GIL changes hands matters more than the system-call interface. A reactor written in C that takes the GIL once per batch of requests, rather than once per request, brought four threads from 0.79× to 2.38× the throughput of one. That study used a synchronous handler written for the benchmark and did not parse HTTP. Its conclusion named the next step: a server that parses HTTP in C and enters Python only to run the application.

This paper takes that step behind the standard interface, ASGI [8], so that real frameworks run unchanged. We ask four questions.

Q1. How much does one request cost when the transport and the parser run outside the GIL, compared with Uvicorn and Granian?

Q2. Does GIL batching still help worker threads when the handler is an ASGI application, which builds a scope and steps a coroutine for each request?

Q3. How long does a real framework hold the GIL, and what does that time allow threads to deliver?

Q4. What changes on the free-threaded build?

The paper makes three contributions. The first is `uringpy-http`, an ASGI server on `io_uring` with picohttpparser [5] running without the GIL. It takes the GIL once per batch of requests and starts each application as an eager `asyncio` task (Section III). The second is a comparison with Uvicorn and Granian [7] that covers three applications, one to four workers, threads and processes, both CPython builds and open-loop load, with expectations committed before the runs (Sections IV and V). The third is a measurement of how long the GIL is held per request inside Starlette and FastAPI, and of what that time limits (Section V-C).

II. Background and Related Work

ASGI. An ASGI application is an `async` callable `app(scope, receive, send)` [8]. The server builds `scope`, a dictionary with the method, path, headers and other details of one request. The application awaits `receive()` for the body and calls `send()` twice, once with the status and headers and once with the body. Starlette [9] and FastAPI [10] are ASGI frameworks. Uvicorn [2] and Hypercorn run ASGI applications on `asyncio`, and Granian [7] runs them on a Rust HTTP stack. In production these servers scale with one process per core.

Parsing. `httptools` wraps llhttp [4], a C parser that re-

*M. Velayutham is an independent researcher (e-mail: marimuthu.vm@gmail.com; ORCID: 0009-0007-7352-5552).

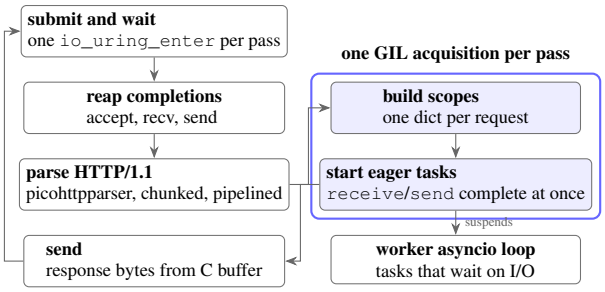


FIGURE 1. One pass of a worker. Plain boxes run in C with the GIL released; the shaded steps run under one acquisition for every request in the pass, or for one request at a time with batching off.

ports a request through callbacks; in Uvicorn each callback is a Python method. h11 is written entirely in Python. picohttpparser [5] parses a whole header block in one call, writes the results into an array the caller provides, and uses SIMD instructions where they are available; H2O [6] uses it. We use picohttpparser because a single call with no callbacks can run with the GIL released.

The GIL and `io_uring`. The GIL lets one thread run Python code at a time. How badly it treats I/O-bound threads is known since Beazley [12], and the effect is an instance of Amdahl's law [13] for contended critical sections [14]. `io_uring` [11] lets a program submit many I/O operations and collect their completions with one system call. The earlier study [1] combined the two. With one Python call per request, throughput is bounded by $X_N \le 1/(t_p + g\tau)$. Here t_p is the time a request holds the GIL, g is the number of GIL hand-offs per request, and τ is the cost of each hand-off. Batching lowers g; it cannot lower t_p. PEP 703 [15] made a build without the GIL available, and it is supported from CPython 3.14.

III. Design

`uringpy-http` is about 1,500 lines of Cython and Python plus the vendored picohttpparser and liburing 2.6. One *engine* is one worker. It owns an `io_uring` ring, a listening socket with `SO_REUSEPORT` and a connection table indexed by file descriptor. Engines share nothing. They run as threads of one interpreter or as one process each. Figure 1 shows one pass over the completion queue.

Transport and parsing without the GIL. Each connection receives into a buffer of its own, which grows as needed. When bytes arrive, the engine calls `phr_parse_request` on the buffer. If the request is complete, it lowercases the header names in place, reads `Content-Length`, `Transfer-Encoding`, `Connection` and `Expect`, decodes a chunked body in place and percent-decodes the path. None of this touches a Python object. A request that is not yet complete re-arms a receive. Malformed or oversized requests get 400, 413, 431 or 501 responses from static buffers, followed by a lingering close. Pipelined requests stay in the buffer and are parsed once the previous response has been sent.

One GIL acquisition per pass. The engine collects every complete request it finds in one pass over the completion queue. It then takes the GIL once to build each request's `scope` and start each application. An option takes the GIL once per request instead; it exists for comparison. The application starts as an `asyncio.Task` with `eager_start=True`, the eager task factory added in Python 3.12. The task runs immediately, in the caller's context, until it first suspends. `receive()` returns an object that is already resolved and holds the body, and `send()` appends status, headers and body to a C buffer owned by the connection and also returns an already-resolved object. A request that never waits on real I/O therefore finishes inside that single acquisition. It still has full `asyncio` semantics: the current task, a fresh `contextvars` context and anyio cancellation scopes. Once the GIL is released, the engine sends the responses that are complete.

Requests that wait. An application that awaits real I/O, such as a database call, `asyncio.sleep` or FastAPI's thread pool for synchronous endpoints, suspends its task on the worker's own `asyncio` loop. The ring polls that loop's selector descriptor, and the ring's wait times out at the loop's next timer. Each pass runs one loop iteration when the loop has work, under the same GIL acquisition. A task's response goes out as soon as it is complete. A connection that closes while its application still runs bumps a generation counter, so a later `send()` raises `ClientDisconnected`, as ASGI 2.4 specifies.

Scope. The server implements HTTP/1.0 and 1.1, keep-alive, pipelining, chunked and `Content-Length` bodies, `100-continue`, HEAD, streaming responses (chunked encoding) and lifespan with state. It does not implement TLS, HTTP/2 or WebSockets, and it does not stream request or response bodies before they are complete. Fifty-one tests, run on CPython 3.12, 3.14 and 3.14t, include unmodified Starlette and FastAPI applications.

IV. Method

Testbed. The server was a Google Compute Engine `t2d-standard-4` (AMD EPYC 7B13, four vCPUs with one thread per core). The load generator ran on a `t2d-standard-8` in the same zone. Both ran Ubuntu 24.04 with kernel 7.0.0-1014-gcp. The earlier study used us-east1-b; on the day of this run neither us-east1 nor us-central1 had capacity for these machine types, so both machines were in us-east4-a. The software was CPython 3.14.8 and 3.14.8t (free-threaded), Uvicorn 0.54.0, `httptools` 0.9.0, `uvloop` 0.23.0, Granian 2.8.4, Starlette 1.8.0, FastAPI 0.143.1 and pydantic 2.14.1.

Servers and applications. Uvicorn ran with `httptools` and `uvloop` (the usual production setup) and with h11 and `asyncio`. In thread mode each worker thread runs its own event loop and Uvicorn server on its own `SO_REUSEPORT` listener, the same layout `uringpy-http` uses. Granian ran in its own process mode. The raw application sends a 13-byte body in two `send` calls. The Starlette application is a route with a typed path parameter that returns JSON. The FastAPI application adds a query parameter and a pydantic response model. Every request was `GET /items/42?q=hello`.

Protocol. Closed-loop load came from wrk with 8 threads

TABLE I. One worker: thousands of requests per second (mean ± 95% half-width) and the ratio to Uvicorn with `httptools`.

	raw ASGI		Starlette		FastAPI	
server	k/s	ratio	k/s	ratio	k/s	ratio
Uvicorn, h11	8.3 ± 0.1	0.28	6.2 ± 0.1	0.42	3.8 ± 0.1	0.64
Uvicorn, httptools	29.8 ± 0.8	1	14.6 ± 0.2	1	5.9 ± 0.1	1
Granian	70.9 ± 1.4	2.38	24.2 ± 0.4	1.66	7.5 ± 0.0	1.28
uringpy-http	$\mathbf{100.1 \pm 2.2}$	**3.36**	$\mathbf{35.1 \pm 0.6}$	**2.41**	$\mathbf{8.6 \pm 0.0}$	**1.47**
no Python	125.9 ± 4.4	4.23				

Ratios are to Uvicorn processes at one worker; `uringpy-http` is GIL per batch, one thread (one process: 100.0, 35.1, 8.7).

TABLE II. Four workers (closed loop, GIL build): threads over processes of the same server (T/P), and per request the voluntary context switches (c.sw.) and GIL owner changes (g) of the threads.

app	server (threads)	T/P	c.sw.	g
raw	Uvicorn, httptools	0.17	4.33	0.352
	uringpy-http, GIL/request	0.38	1.11	0.180
	uringpy-http, GIL/batch	**0.76**	0.021	0.020
	uringpy-http, no Python	1.00	0.000	0.000
Starlette	Uvicorn, httptools	0.20	4.40	0.432
	uringpy-http, GIL/request	0.28	1.03	0.028
	uringpy-http, GIL/batch	0.34	0.021	0.020
FastAPI	Uvicorn, httptools	0.21	4.59	0.729
	uringpy-http, GIL/request	0.25	1.01	0.024
	uringpy-http, GIL/batch	0.26	0.104	0.032

and 400 keep-alive connections; open-loop load came from wrk2 [16] at fixed rates. Every run started a fresh server, warmed it for 5 s and measured for 20 s. Counters were sampled at the start and end of the measured interval and differenced: requests, GIL acquisitions, the GIL owner-change counter g read from the interpreter [1], voluntary context switches and processor time, together with `/proc/stat` on both machines. There were five repetitions (three for GIL timing and open loop), interleaved in a shuffled order. We report means with 95% Student-t half-widths, and ratios with first-order error propagation. The coefficient of variation had a median of 0.8% and a maximum of 3.3%. The expectations, with the thresholds that would refute them, were committed (`bench/EXPERIMENTS.md`) before the first run. Section V-F lists the outcomes.

V. RESULTS

A. The cost of one request (Q1)

Table I compares the servers with one worker. Each server is one event loop on one core, so this is the per-request cost of each design. On the raw application `uringpy-http` serves 100.1k requests/s and Uvicorn with `httptools` 29.7k, 3.36× as many. With Starlette the ratio is 2.41×, and with FastAPI 1.47×. The ratio shrinks as the framework's own Python work grows, because that work runs equally slowly under every server. Granian runs its HTTP stack in Rust and enters Python only for the application. It falls between Uvicorn and `uringpy-http` on all three applications. Answering every request with fixed bytes and never entering Python, `uringpy-http` serves 125.9k/s. Calling the raw ASGI application therefore costs about 2 µs on top of the transport and the parser.

These runs compare whole servers, so they do not separate the parser from the transport. The earlier study measured `io_uring` against `epoll` at 1.15–1.25× for one worker. Most of the difference measured here therefore comes from doing the HTTP work and the loop in C rather than in Python callbacks.

B. Scaling with workers (Q2)

Figure 2 and Table II show one to four workers. With processes, `uringpy-http` scales by 2.57× (raw), 3.04× (Starlette) and 3.74× (FastAPI) and reaches 257.0k, 106.8k and 32.4k requests/s. That is 2.76×, 2.14× and 1.44× Uvicorn's processes. Granian, at 123.4k, 64.0k and 20.5k, reaches 1.32×, 1.29× and 0.91×. On the raw application four processes keep all four cores busy at 3.99 cores, and the server with no Python reaches 307k, so the load generator and the network are not what limits the raw application.

Worker threads tell a different story. Uvicorn's threads deliver less with four workers than with one on every application (S_4 = 0.53, 0.68, 0.81), and each request costs 4.3–4.6 voluntary context switches. Taking the GIL once per request does not rescue `uringpy-http` either. Its four threads reach 0.98 of one thread on the raw application, with 1.1 context switches and 0.18 GIL owner changes per request. Batching cuts those to 0.02 and 0.02 and doubles throughput, to 196.5k (2.09× the per-request mode). That is 0.76 of four processes. We expected at least 0.90, so that expectation is refuted. On the frameworks, batching adds 25% (Starlette) and 7% (FastAPI) over taking the GIL per request, and the threads stay at 0.34 and 0.26 of processes. Latency follows: at four workers the 99th percentile of FastAPI requests is 2.4 s on Uvicorn threads, 238 ms on batched `uringpy-http` threads and 14 ms on its processes.

C. Time under the GIL (Q3)

Why do the frameworks gain so little? With timing turned on, one worker of `uringpy-http` holds the GIL for 2.44 µs per request on the raw application, 20.8 µs on Starlette and 105.6 µs on FastAPI (Table III). That is building the scope, the framework's routing and validation, serialising the response and stepping the coroutine. No number of threads can exceed $1/t_p$ requests per second, and those ceilings are 410k, 48.1k and 9.5k. Batched threads reach 0.48, 0.76 and 0.89 of them. For a framework, the GIL is close to saturated, so the remaining gain from batching, or from anything else done outside the GIL, has to be small: four FastAPI threads would deliver at most 9.5k/s if hand-offs cost nothing, against the 8.4k measured. For the raw application the ceiling is far off, and the 0.76 of processes is limited by something other than t_p. At four threads the raw application's hold time rises to 4.5 µs, which we attribute to contention for the shared interpreter state. That is close to the 4 µs handler at which the earlier study's batched threads fell to 0.63 of processes.

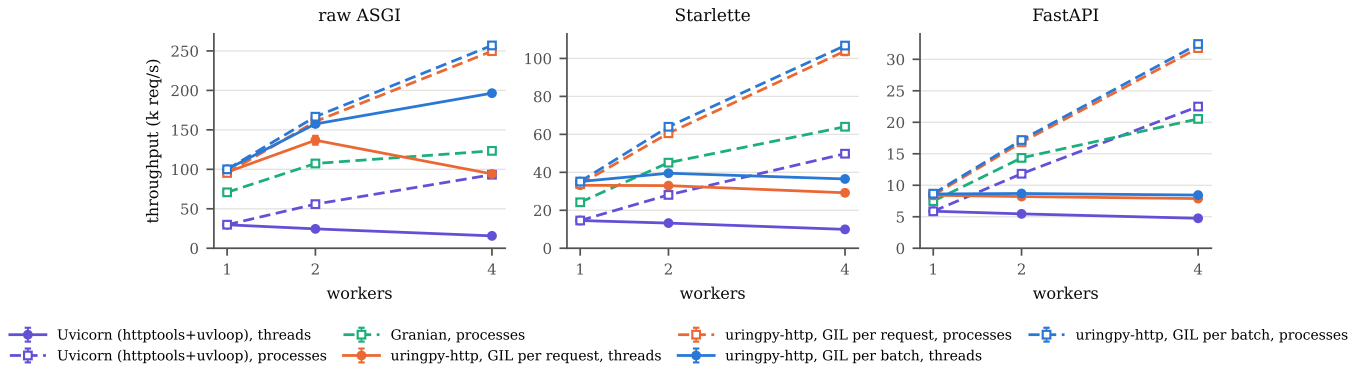


FIGURE 2. Throughput against workers (closed loop, GIL build). Solid lines and filled markers are threads of one interpreter; dashed lines and hollow markers are processes. Error bars are 95% intervals (most are smaller than the markers).

TABLE III. GIL hold per request with one worker, the ceiling it sets on any number of threads, and what four batched threads deliver.

app	t_p (µs)	$1/t_p$ (k/s)	4 threads (k/s)	share
raw ASGI	2.44	410	196.5	0.48
Starlette	20.8	48.1	36.5	0.76
FastAPI	105.6	9.47	8.4	0.89

Per-request mode gives 2.56, 21.9 and 106.5 µs. With batching at four threads the timer brackets a whole batch in wall time, and CPython makes the holder drop the GIL every 5 ms when another thread waits, so batched t_p at $N > 1$ overstates the hold (FastAPI: 248 µs, an impossible utilisation of 2.06); we do not use it.

TABLE IV. Free-threaded CPython 3.14.8t: thousands of requests per second and threads over processes at four workers.

app	server	N=1	N=4	T/P
raw	uringpy-http, GIL/batch	95.8	236.7	0.97
	uringpy-http, GIL/request	91.1	232.3	0.95
	Uvicorn, httptools+asyncio	24.4	68.1	0.91
FastAPI	uringpy-http, GIL/batch	7.1	22.3	0.84
	uringpy-http, GIL/request	6.9	21.5	0.81
	Uvicorn, httptools+asyncio	4.9	16.3	0.87

`uvloop` is not available for this build. Processes: raw 244.6 and 75.2; FastAPI 26.5 and 18.8.

D. The free-threaded build (Q4)

Without the GIL, worker threads scale like processes (Table IV). The interpreter reported the GIL disabled in all 100 server runs. Four batched threads deliver 236.7k on the raw application and 22.3k on FastAPI. That is 0.97 and 0.84 of free-threaded processes, and 1.20× and 2.65× what the same threads deliver on the GIL build. Whether the GIL is taken per batch or per request no longer matters (within 2–4%). The FastAPI result is below the 0.90 we expected, so that part of the expectation is refuted. A single worker pays for the build: 4% on the raw application and 17% on FastAPI. Four free-threaded processes deliver 0.82 of the GIL build's processes on FastAPI. Uvicorn's threads also scale on this build (0.91 and 0.87 of its processes), but they start from a lower point.

E. Open-loop latency

Figure 3 shows tail latency under fixed offered rates with four workers. On the raw application, batched `uringpy-http` processes keep the 99th percentile under 4 ms up to 200k requests/s (median 1.61 ms). Up to 50k requests/s Uvicorn's processes are as fast as the other configurations, but they cannot keep up at 100k. Batched threads keep up to 200k, at a 99th percentile of 0.85 s, and fall behind at 250k. Threads that take the GIL per request fall behind at 100k. On FastAPI, Uvicorn's processes keep up with 20k requests/s, but their 99th percentile reaches 2.0 s. Batched `uringpy-http` processes serve the same rate at 5.7 ms, keep up with 30k, and fall behind at 40k. Threads on FastAPI cannot keep up even with 10k, because that is more than their closed-loop capacity of 8.4k.

F. Expectations and outcomes

Of the fourteen expectations committed before the runs, eleven held. Two were refuted, both about how close threads come to processes: batched threads on the raw application (0.76 against ≥0.90) and free-threaded threads on FastAPI (0.84 against ≥0.90). One could not be tested: open-loop median latency at a quarter of capacity for FastAPI, because the lowest offered rate was already above half the capacity of the thread configurations. Every per-request speedup over Uvicorn that we predicted held, with margin. `paper/OUTCOMES.md` lists all fourteen.

VI. DISCUSSION

Where the time goes. A request served by Uvicorn spends most of its time in the server, not the application. Moving the transport and the parser out of Python makes the raw application 3.36× faster, while FastAPI, which spends about 100 µs of its own per request, gains 1.47×. The gain is largest for small, fast endpoints, such as health checks, proxies and JSON APIs with light validation, and it holds with processes as well as with one worker.

What batching can and cannot do. Batching attacks the hand-offs (g), and it removes nearly all of them: context switches fall from about one per request to two per hundred. That doubles thread throughput when the application is cheap. The ceiling $1/t_p$ does not move, though, and real frameworks sit close to it. On the GIL build, threads are a way to share memory and connection pools, not a way to use more cores. That holds for any server, because t_p belongs to the application.

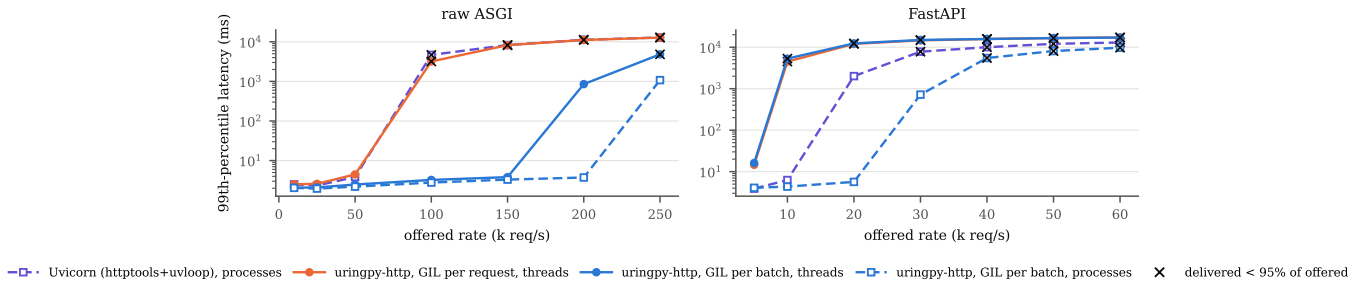


FIGURE 3. 99th-percentile latency under open-loop load (wrk2, corrected for coordinated omission), four workers, log scale. A cross marks a run that delivered less than 95% of the offered rate.

Deployment. On the GIL build, run `uringpy-http` with one process per core. On the free-threaded build, threads are within 3–16% of processes and share one interpreter's memory, at a 4–17% cost per core that should shrink as the build matures. Threads on the GIL build make sense only for applications that hold the GIL for a few microseconds.

VII. Threats to Validity

All runs were on one four-core machine type, in one zone, in one session on one day. The earlier study saw throughput vary by 0.89–1.09× between days, so the absolute numbers carry that uncertainty, though ratios measured in the same session are less affected. The applications are small and do no I/O of their own; an application that waits on a database is served by the worker's loop (Section III), which these benchmarks exercise only in tests. The closed-loop runs used 400 connections. Ten runs, all Uvicorn with FastAPI at one worker, had 32–56 wrk timeouts each in about 117k requests; their slowest responses exceeded the 10 s timeout. We kept these runs in the means. The parser and the transport were not measured separately. Our server implements a subset of HTTP (Section III), and a server that implemented TLS or HTTP/2 would do more work per request. Finally, the batched GIL timer overstates the hold at more than one thread, as Table III notes, so we interpret only the one-worker value.

VIII. Conclusion

Parsing HTTP and driving the socket in C with the GIL released makes a request through a Python ASGI server 1.47–3.36× cheaper than through Uvicorn, and the gain holds at four processes (1.44–2.76×). Batching the GIL removes almost every hand-off between worker threads and doubles their throughput for a light application. It cannot raise the ceiling that the application's own time under the lock sets. For Starlette and FastAPI that ceiling is close, so on the GIL build processes remain the way to use more cores, and on the free-threaded build threads come within 3–16% of them.

Availability. Code, tests, raw run files, the scripts that produced every table and figure, and the expectations recorded before the runs are at `https://github.com/marimuthuvm/uringpy-http` (MIT).

References

[1] M. Velayutham, "The lock, not the system call: What batching buys multithreaded Python servers under the GIL," 2026. Code and data: `https://github.com/marimuthuvm/uringpy`, doi:10.5281/zenodo.23262923.

[2] Encode, "Uvicorn: an ASGI web server," `https://www.uvicorn.org`.

[3] MagicStack, "uvloop: ultra fast asyncio event loop," `https://github.com/MagicStack/uvloop`.

[4] F. Indutny et al., "llhttp: port of http_parser to llparse," `https://github.com/nodejs/llhttp`; Python binding `httptools`, `https://github.com/MagicStack/httptools`.

[5] K. Oku et al., "PicoHTTPParser: a tiny, primitive, fast HTTP request/response parser," `https://github.com/h2o/picohttpparser`.

[6] K. Oku et al., "H2O: the optimized HTTP/1.x, HTTP/2, HTTP/3 server," `https://h2o.example.net`.

[7] G. Barbieri, "Granian: a Rust HTTP server for Python applications," `https://github.com/emmett-framework/granian`.

[8] "ASGI (Asynchronous Server Gateway Interface) specification," version 3.0, HTTP spec 2.4, `https://asgi.readthedocs.io`.

[9] Encode, "Starlette," `https://www.starlette.io`.

[10] S. Ramírez, "FastAPI," `https://fastapi.tiangolo.com`.

[11] J. Axboe, "Efficient IO with io_uring," 2019, `https://kernel.dk/io_uring.pdf`.

[12] D. Beazley, "Understanding the Python GIL," PyCon, 2010.

[13] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[14] S. Eyerman and L. Eeckhout, "Modeling critical sections in Amdahl's law and its implications for multicore design," in *Proc. ISCA*, 2010, pp. 362–370.

[15] S. Gross, "PEP 703: Making the Global Interpreter Lock optional in CPython," 2023, `https://peps.python.org/pep-0703/`.

[16] G. Tene, "wrk2: a constant throughput, correct latency recording variant of wrk," `https://github.com/giltene/wrk2`.